DOCUMENT

12

Specification Pack

Constitution, specification, plan, contract and tasks

Constitution

Core Principles

I. Business Rules Live in the Domain Layer

Every approval rule is implemented in `backend/src/invoice_app/domain/` and nowhere else. Routers translate HTTP to domain calls and domain errors to status codes; they never decide anything. `main.py` wiring contains no business logic.

II. Test-First for Rules, With Negative Cases (NON-NEGOTIABLE)

Every rule ships with tests written before or with the code. Each "may not" rule has a negative test that proves two things: the attempt is refused with a clear reason, **and the invoice is byte-for-byte unchanged afterwards**. A positive-path test alone never counts as covering a rule.

III. Boundaries Are Tested On Both Sides

For each numeric limit, tests cover the value at the limit, one cent above and one cent below. Limits are `Decimal`, never `float`.

IV. Refusals Explain Themselves

A refused action returns a human-readable reason that names the rule and the actor. Reasons are asserted in tests, so wording changes are deliberate.

V. Mutation Score Is Evidence, Not a Target

Unit tests are judged by `mutmut` on the domain package. Surviving mutants are triaged: close the gap with a test, or record why the mutant is equivalent or cosmetic.

VI. Small, Reviewable Steps

Features land in independent steps, each with its own tests and commit message that says what changed and why. Every review finding on a PR gets a verdict: fixed, dismissed with a reason, or deferred with a reason.

Additional Constraints

- Stack: Python 3.10+, FastAPI, pytest, Playwright for end-to-end. In-memory storage only.
- End-to-end tests use page objects and role-based locators (`getByRole`, `getByLabel`), never CSS or XPath.
- Never commit real credentials. Demo accounts are local and documented as such.

Development Workflow

Spec (Spec Kit) → plan → tasks → implement in steps → unit tests reviewed by the `test-reviewer` agent → mutation run → e2e → PR reviewed by the `pr-reviewer` agent. All AI interactions are logged to `logs/ai-interactions.jsonl` by project hooks.

Governance

This constitution supersedes ad-hoc preferences. Amendments are made in a commit that states what changed and why. `AGENTS.md` is the runtime guidance file for agents and must stay consistent with this document.

Version: 1.0.0 | **Ratified**: 2026-10-16 | **Last Amended**: 2026-10-16

Feature specification

Feature Branch: `001-invoice-approval-limits`

Created: 2026-10-16

Status: Implemented

Input: User description: "Add approval limits by role to the invoice approval service. A Manager may approve invoices up to 50,000. A Director may approve invoices up to 500,000. Anything above 500,000 needs two distinct approvers, and the second approver must be a director. The same user may never count as both approvers on one invoice. A rejected attempt returns a clear reason, and nothing about the invoice changes."

Clarifications

Session 2026-10-16

- Q: Are the limits inclusive? → A: Yes. "Up to 50,000" means 50,000.00 is allowed and 50,000.01 is not. Likewise 500,000.00 is a single-Director invoice and 500,000.01 needs two approvers.
- Q: Who may give the *first* approval above 500,000? → A: A Manager or a Director. The requirement only constrains the second approver. (Ambiguity flagged; this is the least-restrictive reading of the text.)
- Q: What does "the same user" mean? → A: The same account, compared after trimming whitespace and ignoring case, so `Dir.One` and `dir.one` are one user.
- Q: Which check wins when the same Manager tries to approve twice? → A: The duplicate-user reason is reported (it is the more specific problem).
- Q: What happens to the legacy `Approver` role? → A: It is replaced by `Manager`. `Admin` keeps destructive and reject powers but has **no approval authority**.
- Q: What does "a rejected attempt" mean — the existing Reject action? → A: No. It means *an approval attempt that the rules refuse*. The existing explicit Reject action is unchanged.
- Q: Does a partial (first) approval count as a change that a later refusal might undo? → A: A refused attempt never writes anything. A first approval that was accepted stays recorded.

User Scenarios & Testing *(mandatory)*

User Story 1 - Limit each approver to their role's amount (Priority: P1)

A Manager can approve everyday invoices; larger ones need a Director. People should learn immediately why a request was refused.

Why this priority: It is the core control. Without it every approver can approve any amount.

Independent Test: Approve invoices of 50,000.00, 50,000.01 and 500,000.00 as a Manager and as a Director; check results and that refused attempts leave the invoice untouched.

Acceptance Scenarios:

1. **Given** a pending invoice of 50,000.00, **When** a Manager approves it, **Then** it becomes Approved and records the Manager as decider.

2. **Given** a pending invoice of 50,000.01, **When** a Manager approves it, **Then** the attempt is refused with a reason naming the 50,000.00 Manager limit and the invoice is unchanged.
3. **Given** a pending invoice of 500,000.00, **When** a Director approves it, **Then** it becomes Approved with that single approval.
4. **Given** a pending invoice of any amount, **When** an Admin (or any role that is not Manager/Director) tries to approve it, **Then** it is refused with a reason and nothing changes.

User Story 2 - Two distinct approvers above 500,000 (Priority: P1)

Very large invoices need a second pair of eyes, who must be a Director and a different person.

Why this priority: Highest-risk invoices; this is where a plausible-looking implementation most often breaks (e.g. one user approving twice).

Independent Test: Create a 600,000.00 invoice; approve with Director A (stays Pending, one approval), approve again with Director A (refused), with a Manager (refused), then with Director B (Approved).

Acceptance Scenarios:

1. **Given** a pending invoice of 500,000.01, **When** a Director approves it, **Then** it stays Pending Approval with exactly one recorded approval and no decider.
2. **Given** that invoice with one approval, **When** a *different* Director approves, **Then** it becomes Approved and both approvals are recorded.
3. **Given** that invoice with one approval by user X, **When** X approves again (including `X` in different case or with surrounding spaces), **Then** it is refused with a reason and the invoice still has exactly one approval.
4. **Given** that invoice with one approval, **When** a Manager tries to give the second approval, **Then** it is refused with a reason saying the second approver must be a Director.
5. **Given** a pending invoice of 500,000.01, **When** a Manager gives the first approval and a Director the second, **Then** it becomes Approved.

User Story 3 - Refusals are visible in the UI and change nothing (Priority: P2)

A user who clicks Approve on an over-limit invoice sees the reason on screen and the invoice stays pending.

Why this priority: The rule is only useful if people can see why; also proves the rule through the real UI.

Independent Test: Playwright: sign in as the demo Manager, click Approve on the 75,000.00 invoice, read the alert, confirm the invoice is still Pending Approval.

Acceptance Scenarios:

1. **Given** the signed-in demo Manager and a pending 75,000.00 invoice, **When** they click Approve, **Then** an alert shows the Manager-limit reason and the invoice remains in the pending list and shows "Pending Approval".
2. **Given** a pending invoice above 500,000, **When** the first approval is recorded, **Then** the pending list shows "1 of 2 approvals".

Edge Cases

- Exactly 50,000.00, 50,000.01, 500,000.00 and 500,000.01 (inclusive limits; one cent either side).
- Same user with different case/whitespace as the second approver.
- Second approver is a Manager; first approver is a Manager.
- Approving an invoice that is already Approved, Rejected or Draft (existing state errors still apply; nothing changes).
- A refused attempt followed by a valid attempt by someone else still works.
- Explicitly rejecting a partially approved invoice still works.
- Blank username on an approver is invalid.

Requirements *(mandatory)*

Functional Requirements

- **FR-001**: A Manager MUST be able to approve an invoice whose amount is ≤ 50,000.00.
- **FR-002**: A Director MUST be able to approve an invoice whose amount is ≤ 500,000.00.
- **FR-003**: An invoice with amount > 500,000.00 MUST require two approvals from two distinct users; it stays Pending Approval after the first and becomes Approved after the second.
- **FR-004**: The second approver of an invoice > 500,000.00 MUST have the Director role. The first MAY be a Manager or a Director.
- **FR-005**: The same user MUST NOT count as both approvers; usernames are compared trimmed and case-insensitively.
- **FR-006**: Roles other than Manager and Director MUST NOT be able to approve any invoice.
- **FR-007**: A refused approval MUST return a clear, specific reason (HTTP 403 with `{"error": "<reason>"}`) and MUST leave the invoice completely unchanged (status, decider, decision time, recorded approvals).
- **FR-008**: Each accepted approval MUST be recorded with the approver's username, role and a UTC timestamp.
- **FR-009**: Existing behaviours (submit, explicit reject with required reason, 404/409/400 mapping, Admin-only delete) MUST keep working.
- **FR-010**: The UI MUST show a refusal reason and, for invoices needing two approvals, how many approvals are recorded.

Key Entities

- **Approver**: username + role of the person acting. Username must not be blank.
- **ApprovalRecord**: one accepted approval (username, role, UTC time) stored on the invoice.
- **Invoice**: existing entity, now carrying an ordered list of ApprovalRecords; `requiredApprovals` is derived from its amount.
- **ApprovalPolicy**: pure rules deciding allowed / refused-with-reason from amount, approver and earlier approvals.

Success Criteria *(mandatory)*

Measurable Outcomes

- **SC-001**: Every rule in FR-001..FR-007 has at least one passing positive test and, for each "MUST NOT" rule, a negative test asserting both the reason and an unchanged invoice.
- **SC-002**: All four limit boundaries (50,000.00 / .01 and 500,000.00 / .01) are covered on both sides.
- **SC-003**: Mutation score of the domain package does not regress and ends at ≥ 90% with every survivor explained.
- **SC-004**: One Playwright test demonstrates a refused over-limit approval through the UI.

Assumptions

- Currency is a single unspecified unit; amounts are exact decimals.
- Sign-in and roles come from the existing demo accounts (`demo.manager` , `demo.director` , `demo.director2` , `demo.admin`); there is no user management.
- Storage stays in memory; concurrency beyond the existing repository lock is out of scope (see deferred review finding).
- Notification of the first approver or of Directors is out of scope.

Implementation plan

Branch: `001-invoice-approval-limits` | **Date**: 2026-10-16 | **Spec**: spec.md

Input: Feature specification from `/specs/001-invoice-approval-limits/spec.md`

Summary

Add a pure `approval_policy` module to the domain layer that decides, from an invoice amount, the approver and earlier approvals, whether an approval is allowed and why not. `ApprovalService.approve` calls it before mutating anything, records an `ApprovalRecord`, and only marks the invoice Approved once the required number of approvals is reached. The API maps the new `ApprovalNotPermittedError` to HTTP 403 with the reason. The UI shows approval progress.

Technical Context

Language/Version: Python 3.10+ (developed on 3.13; mutation runs on 3.10 in WSL)

Primary Dependencies: FastAPI, uvicorn; dev: pytest, httpx, mutmut

Storage: In-memory repository (unchanged)

Testing: pytest (unit + API), mutmut (mutation), Playwright (one e2e)

Target Platform: Local dev server, Windows/Linux

Project Type: web-service with a static UI

Performance Goals: N/A (demo app)

Constraints: Domain stays free of web/storage code; refused attempts must be side-effect free

Scale/Scope: One service method, one policy module, one error type, one UI line

Constitution Check

Principle	Status
I. Rules in domain	Pass: `approval_policy.py` + `approval_service.py`; router only maps errors
II. Negative tests assert unchanged invoice	Pass: shared `assert_refused_and_unchanged` helper in service tests
III. Boundaries both sides	Pass: parametrized limits at ±0.01
IV. Refusals explain themselves	Pass: reasons asserted verbatim
V. Mutation evidence	Pass: before/after in `docs/mutation/`
VI. Small steps	Pass: two implementation steps (single-approver, then two-approver), see tasks

Project Structure

Documentation (this feature)

```
specs/001-invoice-approval-limits/
├── spec.md
├── plan.md
├── research.md
├── data-model.md
├── quickstart.md
├── contracts/approvals-api.md
├── checklists/requirements.md
└── tasks.md
```

Source Code (repository root)

```
backend/
├── src/invoice_app/
│   ├── domain/
│   │   ├── approval.py            # Approver, ApprovalRecord (new)
│   │   ├── approval_policy.py     # pure limits + two-approver rules (new)
│   │   ├── approval_service.py    # approve() uses the policy, records approvals (changed)
│   │   ├── errors.py              # + ApprovalNotPermittedError (changed)
│   │   └── invoice.py             # + approvals list (changed)
│   └── api/
│       ├── auth/demo_users.py     # + Manager, Director accounts (changed)
│       ├── main.py                # 403 mapping (changed)
│       ├── routers/approvals.py   # passes Approver(username, role) (changed)
│       ├── serialization.py       # + approvals, requiredApprovals (changed)
│       └── wwwroot/app.js         # "1 of 2 approvals" (changed)
└── tests/
    ├── test_approval_policy.py    # pure rule tests (new)
    ├── test_approval_service.py   # service + unchanged-on-refusal tests (changed)
    └── test_api.py                # HTTP-level 403 / two-approver flow (changed)
e2e/
├── pages/{LoginPage,ApprovalsPage}.ts
└── tests/over-limit-rejection.spec.ts
```

Structure Decision: keep the existing Domain / Api / Tests layering of the original C# solution, ported to Python packages.

Complexity Tracking

No constitution violations.

Research and decisions

Decision	Choice	Rationale	Rejected alternative
Money type	`decimal.Decimal`	Exact comparison at 50,000.00 / .01 boundaries	`float` (0.01 steps are inexact)
Where rules live	Pure function `rejection_reason(amount, approver, prior)`	Easy to unit test and mutation-test without a repository	Logic inside `ApprovalService.approve`
Refusal signal	`ApprovalNotPermittedError(reason)` → 403	Distinct from 409 (wrong state) and 400 (bad input)	Reusing `InvalidStateError`
No partial writes	Check first, then mutate	The in-memory repository returns the same object; mutating before checking would leak state	Roll back after failure
"Same user"	`strip().casefold()`	Stops `Dir.One` vs `dir.one` bypass	Exact string equality
Approvals storage	`list[ApprovalRecord]` replaced, not appended in place	A new list is assigned only after all checks pass	In-place append
Mutation tool	`mutmut` 3 in WSL	Needs `fork()`; unavailable natively on Windows	cosmic-ray (heavier config)
Python port	FastAPI	Mirrors ASP.NET controller/service split	Flask (less typing support)

Data model

Approver (value object, not stored)

- `username: str` (non-blank)
- `role: str` (`Manager`, `Director`; anything else cannot approve)

ApprovalRecord (value object, stored on the invoice)

- `username: str`
- `role: str`
- `approved_at: datetime` (UTC)

Invoice (changed)

Existing fields plus `approvals: list[ApprovalRecord]` (ordered, oldest first).

Derived: `required_approvals = 2 if amount > 500000 else 1`.

State transitions

```
Draft --submit--> PendingApproval --approve (count < required)--> PendingApproval
                                  --approve (count == required)--> Approved
                                  --reject(reason)--> Rejected
```

A refused approval is not a transition: no field changes.

Limits

Constant	Value
`MANAGER_LIMIT`	50000
`DIRECTOR_LIMIT`	500000

API contract

Auth: bearer token. Allowed session roles at the router: `Manager`, `Director`, `Admin` (Admin is then refused by the domain with a reason).

Outcome	Status	Body
Approval recorded, more approvals needed	200	invoice JSON, `status: "PendingApproval"`, `approvals` has 1 entry, `requiredApprovals: 2`
Approval recorded, invoice complete	200	invoice JSON, `status: "Approved"`, `decidedBy` = final approver
Approver not allowed by the rules	403	`{"error": "<reason>"}`; invoice unchanged
Invoice not pending	409	`{"error": "Invoice INV-x is Approved; only pending invoices can be decided."}`
Unknown invoice	404	`{"error": "Invoice INV-x was not found."}`
No/invalid token	401	empty

Invoice JSON additions

```
{
  "requiredApprovals": 2,
  "approvals": [
    { "username": "demo.director", "role": "Director", "approvedAt": "2026-10-16T10:00:00+00:00"
}
  ]
}
```

Reason texts (asserted in tests)

- `Role {role} is not allowed to approve invoices.`
- `A Manager may approve invoices up to 50,000.00; this invoice is {amount} and needs a Director.`
- `{username} has already approved this invoice; invoices above 500,000.00 need two different approvers.`
- `The second approver on an invoice above 500,000.00 must be a Director; {username} is a {role}.`

Quickstart

```
pip install -e "backend[dev]"
python -m pytest backend/tests -q                       # unit + API tests
python -m uvicorn invoice_app.api.main:app --app-dir backend/src --port 5000
cd e2e && npm install && npx playwright install chromium && npm test
wsl bash scripts/run_mutation.sh after                  # mutation report ->
docs/mutation/after.txt
```

Manual check (UI at http://localhost:5000):

1. Sign in as `demo.manager` / `ChangeMe123!`, Approvals tab, Approve `INV-2001` (75,000.00) → alert with the Manager-limit reason; invoice stays pending.
2. Sign in as `demo.director`, Approve `INV-2001` → Approved.
3. As `demo.director`, Approve `INV-2002` (600,000.00) → still pending, "1 of 2 approvals". Approve again → refused (already approved). Sign in as `demo.director2`, approve → Approved.

Requirements checklist

Purpose: Validate specification completeness before planning **Created**: 2026-10-16 **Feature**: spec.md

Content Quality

- [x] No implementation details in requirements (language, frameworks)
- [x] Focused on user value and business rules
- [x] All mandatory sections completed

Requirement Completeness

- [x] No [NEEDS CLARIFICATION] markers remain (all resolved in the Clarifications section)
- [x] Requirements are testable and unambiguous
- [x] Success criteria are measurable
- [x] Edge cases identified (boundaries, case/whitespace users, manager as second approver)
- [x] Scope bounded; assumptions listed

Feature Readiness

- [x] Every functional requirement has acceptance scenarios
- [x] User stories cover the single-approver and two-approver flows plus the UI refusal

Notes

- The "first approver may be a Manager" reading is an explicit clarification, not an assumption hidden in code.

Task list

Input: Design documents from `/specs/001-invoice-approval-limits/`

Prerequisites: plan.md, spec.md, research.md, data-model.md, contracts/

Tests: Required (constitution principle II).

Format: `[ID] [P?] [Story] Description`

Phase 0: Baseline (Python port, behaviour-identical)

- [X] T001 Port Domain, Api and tests of the C# demo app to `backend/src/invoice_app` and `backend/tests`; 7 original tests pass as pytest
- [X] T002 [P] Record baseline mutation score (`docs/mutation/baseline.txt`)

Phase 1: Foundational

- [X] T003 [P] `ApprovalNotPermittedError` in `backend/src/invoice_app/domain/errors.py`; map to 403 in `api/main.py`
- [X] T004 [P] `Approver` and `ApprovalRecord` in `backend/src/invoice_app/domain/approval.py`
- [X] T005 `Invoice.approvals` list in `domain/invoice.py`
- [X] T006 Demo accounts `demo.manager`, `demo.director`, `demo.director2` in `api/auth/demo_users.py`; router allows Manager/Director/Admin
- [X] T007 Seed `INV-2001` (75,000.00) and `INV-2002` (600,000.00) in `api/in_memory_repository.py`

Phase 2: User Story 1 - Single-approver limits (P1) — implementation step 1

- [X] T008 [US1] Tests first: Manager at 50,000.00 ok, at 50,000.01 refused; Director at 500,000.00 ok; Admin refused; each refusal asserts reason + invoice unchanged (`tests/test_approval_policy.py`, `tests/test_approval_service.py`)
- [X] T009 [US1] `approval_policy.rejection_reason` single-approver branch + `required_approvals`
- [X] T010 [US1] `ApprovalService.approve(invoice_id, approver)` checks the policy before any mutation and records an `ApprovalRecord`
- [X] T011 [US1] API tests for 403 + reason and unchanged invoice (`tests/test_api.py`)

Phase 3: User Story 2 - Two distinct approvers above 500,000 (P1) — implementation step 2

- [X] T012 [US2] Tests first: first approval keeps Pending; same user (incl. case/whitespace) refused; Manager as second refused; distinct Director completes; refused attempt leaves exactly one approval
- [X] T013 [US2] Extend `rejection_reason` with the duplicate-user and second-must-be-Director rules; `is_same_user` normalisation
- [X] T014 [US2] Service marks Approved only when `len(approvals) >= required_approvals(amount)`; `decided_by` / `decided_at` set on the final approval
- [X] T015 [US2] Serialize `approvals` and `requiredApprovals` in `api/serialization.py`

Phase 4: User Story 3 - Refusal visible in the UI (P2)

- [X] T016 [US3] Show "N of M approvals" in `wwwroot/app.js`
- [X] T017 [US3] Playwright page objects `e2e/pages/*.ts` and test `e2e/tests/over-limit-rejection.spec.ts`

Phase 5: Polish & evidence

- [X] T018 Run `test-reviewer` agent over generated tests; close gaps (non-persisting fake repository, unasserted messages and timestamps)
- [X] T019 Run mutmut again (`docs/mutation/after.txt`); triage survivors
- [X] T020 Update `AGENTS.md`, README, CI to Python
- [X] T021 Review the diff with `pr-reviewer`; record verdicts (`docs/pr-review.md`)
- [X] T021a Fix review findings F1-F3, F5, F6, F8 (atomic approvals, amount validation, `DomainValidationError`, strict `update`); tests written to fail without the fixes
- [ ] T022 Record the demo video and add its link to `ASSIGNMENT.md`
- [ ] T023 Commit in steps, push to the fork, open the PR with `docs/pr-review.md` as the description
- [ ] T024 (deferred F7) Second Playwright spec for the two-Director flow, with a reset endpoint so mutating UI tests are repeatable
- [ ] T025 (deferred F11) Session expiry and constant-time password comparison

Dependencies

Phase 1 blocks Phases 2-3. Phase 3 builds on Phase 2. Phase 4 needs the API shape from Phase 3.